

Capitolo 7a - Esempi di sincronizzazione

Status Done

Problema del buffer limitato

Questo problema illustra la sincronizzazione tra un processo produttore, che produce informazioni, e un processo consumatore, che le consuma. I due processi condividono un buffer di dimensione fissa. La soluzione a questo problema tipicamente usa tre semafori:

1. `mutex`: un semaforo binario inizializzato a 1 per garantire l'esclusione mutua sull'accesso alla regione critica del buffer.
2. `empty`: un semaforo di conteggio inizializzato al numero totale di slot del buffer (N), che conta gli slot vuoti (gestito dal produttore).
3. `full`: un semaforo di conteggio inizializzato a 0, che conta gli slot pieni (gestito dal consumatore).

```
int n;  
semaphore mutex = 1;  
semaphore empty = n;  
semaphore full = 0;
```

Struttura processo produttore:

```
while (true) {  
    ...  
    /* produce an item in next_produced */  
    ...  
    wait(empty);  
    wait(mutex);  
    ...  
    /* add next produced to the buffer */  
    ...  
    signal(mutex);  
    signal(full);  
}
```

▼ Spiegazione codice

- Attesa spazio: il produttore invoca `wait()` sul semaforo `empty`. Il semaforo conta gli slot liberi nel buffer. L'operazione `wait(empty)` decrementa il conteggio degli slot vuoti. Se il conteggio è 0 (buffer pieno), il produttore viene bloccato finché il consumatore non libera uno slot.
- Attesa esclusione mutua: il produttore invoca `wait()` sul semaforo `mutex` (inizializzato a 1). Questa operazione garantisce l'esclusività d'accesso al buffer, prevedendo che un altro processo (sia esso un altro produttore o il consumatore) acceda o modifichi il buffer contemporaneamente (cioè previene una race condition).
- Rilascio esclusione mutua: il produttore invoca `signal()` sul semaforo `mutex`, rilasciando l'accesso esclusivo al buffer. Un eventuale processo in attesa su `mutex` può ora procedere.
- Segnalazione elemento prodotto: il produttore invoca `signal()` sul semaforo `full`. Il semaforo conta gli elementi nel buffer. L'operazione incrementa il conteggio degli slot pieni, segnalando che un nuovo elemento è disponibile per il consumatore (un'operazione necessaria se il consumatore era bloccato in attesa di un elemento).

Struttura processo consumatore:

```
while (true) {  
    wait(full); // full conta gli slot pieni, se il conteggio è 0 => consumatore bloccato finché il p  
    rodotto non: signal(full)  
    wait(mutex); // esclusività d'accesso al buffer condiviso => il consumatore ottiene il lock per a  
    ccedere alla sezione critica  
    ...
```

```

/* remove an item from buffer to next_consumed */
...
signal(mutex); // rilascia il lock per l'accesso al buffer
signal(empty); // incrementa il conteggio degli slot vuoti, segnalando nuovo spazio disponibile p
er il produttore
...
/* consume the item in next consumed */
}

```

Problema dei Lettori-Scrittori

Più processi condividono un insieme di dati. I lettori possono accedere ai dati in modo concorrente, ma gli scrittori devono avere accesso esclusivo. L'obiettivo è prevenire che un lettore e uno scrittore (o più) accedano contemporaneamente.

- La soluzione tipica impiega un semaforo, `rw_mutex`, per l'accesso esclusivo degli scrittori e una variabile `read_count` per contare i lettori attivi.

I processi lettori condividono le seguenti strutture dati:

```

semaphore rw_mutex = 1; // comune sia ai processi lettori che scrittori
semaphore mutex = 1;
int read_count = 0;

```

- `mutex` assicura la mutua esclusione al momento dell'aggiornamento di `read_count`.
- `rw_mutex` è un semaforo di mutua esclusione per gli scrittori e serve anche al primo e all'ultimo lettore che entra o esce dalla sezione critica.

Struttura processo scrittore:

```

while (true) {
    wait(rw_mutex); // blocca lo scrittore se un qualsiasi altro processo detiene già il lock
    ...
    /* writing is performed */
    ...
    signal(rw_mutex); // rilascia il lock e sblocca un eventuale processo in attesa su rw_mutex
}

```

Struttura processo lettore:

```

while (true) {
    wait(mutex); // mutex (sem bin) è usato x proteggere la variabile read_count
    read_count++;
    if (read_count == 1) /* first reader */
        wait(rw_mutex); // il primo lettore blocca tutti gli scrittori in attesa prendendo il lock
    signal(mutex); // lock sul contatore rilasciato x permettere ad altri lettori di aggiornare read_
count
    ...
    /* reading is performed */
    ...
    wait(mutex); // riacquisizione del lock per proteggere read_count prima di decrementarla
    read_count--;
    if (read_count == 0) /* last reader */
        signal(rw_mutex); // sblocca eventuali scrittori in attesa su rw_mutex permettendogli di proc
edere con la scrittura
    signal(mutex); // lock su mutex rilasciato
}

```

Varianti del problema

- Prima variante (priorità ai lettori): nessun lettore deve attendere se non c'è uno scrittore attivo. Questa soluzione può portare al blocco per inedia (starvation) degli scrittori, che potrebbero non avere mai accesso se c'è un flusso costante di lettori.

2. Seconda variante (priorità agli scrittori): se uno scrittore attende, non possono accedere nuovi lettori. Questa soluzione favorisce gli scrittori, risolvendo la loro potenziale starvation, ma può portare a starvation dei lettori.

Il problema dei filosofi a cena

Ci sono cinque filosofi seduti attorno a un tavolo. Tra ogni coppia di filosofi c'è una singola bacchetta, ci sono quindi cinque filosofi e cinque bacchette. Ogni filosofo alterna tra pensare e mangiare. Per mangiare, un filosofo deve prendere entrambe le bacchette adiacenti (quelle alla sua sinistra e quella alla sua destra).

Le bacchette rappresentano le risorse di sistema necessarie per l'esecuzione, come le periferiche o i segmenti di memoria.

La sfida principale è trovare un protocollo di sincronizzazione che eviti il deadlock, ovvero una situazione in cui tutti i processi attendono infinitamente.

- Se tutti e cinque i filosofi hanno fame contemporaneamente e ognuno prende la bacchetta alla sua sinistra, ogni filosofo detiene una risorsa e aspetta l'altra detenuta dal vicino. Nessuno può ottenere la seconda bacchetta, quindi nessuno può mangiare e rilasciare le risorse.

Per eliminare il rischio di stallo, il protocollo di sincronizzazione deve infrangere almeno una delle quattro condizioni di deadlock:

- mutua esclusione,
- possesso e attesa,
- assenza di prelazione,
- attesa circolare.

Soluzione con uso di semafori

Ogni bacchetta è rappresentata con un semaforo, un filosofo cerca di afferrare una bacchetta attraverso la `wait()` su quel semaforo e la posa con `signal()`.

Dati condivisi:

```
semaphore chopstick[5]; // tutti gli elem sono inizializzati a 1
```

Struttura del filosofo:

```
while (true) {
    wait(chopstick[i]);

    wait(chopstick[(i+1) % 5]);
    ...
    /* mangia */
    ...
    signal(chopstick[i]);

    signal(chopstick[(i+1) % 5]);
    ...
    /* pensa */
    ...
}
```

Questa soluzione garantisce che due vicini non mangino contemporaneamente ma non esclude la possibilità di stallo (es. hanno tutti fame contemporaneamente e gli elementi di chopstick diventano tutti 0). Per evitarlo:

- solo quattro filosofi possono stare contemporaneamente a tavola;
- un filosofo può prendere le sue bacchette solo se sono entrambi disponibili;
- soluzione asimmetrica: un filosofo dispari prende prima la bacchetta di sinistra e poi quella di destra, invece un filosofo pari prende prima la bacchetta di destra e poi quella di sinistra.

Soluzione per mezzo di monitor

Utilizzare un monitor per garantire che un filosofo possa prendere le sue bacchette solo quando entrambi disponibili contemporaneamente.

Serve distinguere i tre diversi stati in cui un filosofo può trovarsi:

```
enum {THINKING, HUNGRY, EATING} state[5];
```

Il filosofo i può impostare la variabile a `EATING` solo se i suoi due vicini non stanno mangiando:

```
((state[(i+4) % 5] != EATING) && (state[(i+1) % 5] != EATING))
```

Inoltre c'è:

```
condition self[5]; // permette al filosofo i di ritardare se stesso quando ha fame ma non riesce ad o  
ttenere le bacchette
```

Implementazione monitor: (distribuzione delle bacchette)

```
monitor DiningPhilosophers {  
    enum {THINKING, HUNGRY, EATING} state[5];  
    condition self[5];  
    void pickup(int i) {  
        state[i] = HUNGRY;  
        test(i);  
        if (state[i] != EATING)  
            self[i].wait();  
    }  
    void putdown(int i) {  
        state[i] = THINKING;  
        test((i + 4) % 5);  
        test((i + 1) % 5);  
    }  
    void test(int i) {  
        if ((state[(i + 4) % 5] != EATING) &&  
            (state[i] == HUNGRY) &&  
            (state[(i + 1) % 5] != EATING)) {  
            state[i] = EATING;  
            self[i].signal();  
        }  
    }  
}  
Initialization_code() {  
    for (int i = 0; i < 5; i++)  
        state[i] = THINKING;  
}
```

Per quanto riguarda i filosofi:

```
DiningPhilosophers.pickup(i);  
/* EAT */  
DiningPhilosophers.putdown(i);
```